National University of Computer & Emerging Sciences (FAST–NUCES)

Course: **Agentic AI**

Assignment 1: CrisisSim — Agentic AI for Disaster Response (LLM-based)

1 Project Overview

You are provided with a *minimal simulation framework* of an earthquake disaster scenario. Multiple heterogeneous agents (medics, trucks, drones, survivors, hospitals, depots, rubble, fires) operate on a grid world.

Your task is to **transform this starter code into a complete LLM-based agentic system**. Every major requirement is specified explicitly below.

2 Mandatory Requirements (What You Must Do)

1. **Implement 3 distinct LLM-based reasoning frameworks.**
   - Choose any 3 from: *ReAct*, *Reflexion*, *Plan-and-Execute*, *Chain-of-Thought (CoT)*, *Tree-of-Thought (ToT)*.
   - All reasoning must pass through `reasoning/llm_client.py` to call an LLM API (Groq, Gemini, or equivalent).
   - **No `if/else` rule-based decision logic is allowed.**
   - Planners must output JSON strictly following the **Action JSON Schema** (Section 4).
2. **Extend the environment with at least 3 realistic features**. Examples:
   - **Battery system** for drones/trucks (must recharge at depots).
   - **Medic slowdown** when carrying survivors.
   - **Truck rubble clearing** before moving through blocked roads.
   - **Hospital triage** (capacity-limited queues; FIFO or deadline-based).
   - **Aftershocks** that spawn new rubble or fires at random intervals.
3. **Upgrade the GUI (`server.py`)** to:
   - Display **all entities** with unique shapes/colors: Drone, Medic, Truck, Survivor, Hospital, Fire, Rubble, Depot.
   - Add a **stats panel** showing each tick: survivors remaining, carried, in queues, rescued, deaths, fires extinguished, rubble cleared, battery recharges, hospital overflow events.
   - Add **charts** plotting rescues, deaths, fires extinguished over time, and **stop updating** when the termination condition is met.
4. **Logging & Results Collection**:
   - Log **every prompt/response** to `logs/strategy=<name>/run=<id>/tick=<t>.jsonl`.
   - Save per-run metrics JSON to `results/raw/`.
   - Aggregate results into `results/agg/summary.csv`.
   - Generate required plots (Section 5).
5. **Run Experiments (minimums)**:
   - **3 maps**: `map_small.yaml`, `map_medium.yaml` (you create), `map_hard.yaml`.
   - **5 random seeds per map–strategy**.

- **3 strategies**: your implemented LLM planners.

6. **Write a 6–8 page academic-style report** (Section 7).

3 Repository Structure (What to Edit, Where)

```
crisis-sim/
|   main.py                 # Run single experiments (headless)
|   server.py              # GUI visualization (extend legend, stats, charts)
|   requirements.txt
|
+-- configs/                # YAML maps (add at least map_medium.yaml)
|
+-- env/
|  agents.py  # DroneAgent, MedicAgent, TruckAgent, Survivor (execute commands only)
|  dynamics.py  # Fire spread, aftershocks, rubble clearing, hospital service/triage
|  sensors.py  # World -> compact JSON context for the LLM (bounded token budget)
|   world.py   # CrisisModel (grid, schedule, step loop, termination, metrics)
|
+-- reasoning/
|   llm_client.py           # Implement actual LLM API calls (Groq/Gemini)
|  planner.py           # Dispatch: make_plan(context, strategy, scratchpad)
|   react.py                # ReAct (LLM-based)
|   reflexion.py            # Reflexion
|   # add: plan_execute.py, cot.py, tot.py as needed
|
+-- tools/
|   routing.py              # Navigation helpers (BFS/Dijkstra)
|   hospital.py            # Queue policies: FIFO / earliest-deadline-first
|   resources.py            # Battery/water/tool usage and resupply
|
+-- eval/
|  harness.py           # Batch runs: (map x seed x strategy) -> results/raw
|   plots.py                # Build figures from results/agg/summary.csv
|
+-- logs/                   # LLM prompt/response logs (populate per tick)
\-- results/                # raw/ JSONs, agg/ CSV, plots/ figures
```

4 Action JSON Schema

Every planner must output JSON *exactly* like:

```
{
  "commands": [
    {"agent_id": "2", "type": "move", "to": [5, 7]},
    {"agent_id": "3", "type": "act", "action_name": "pickup_survivor
       "},
    {"agent_id": "4", "type": "act", "action_name": "
       drop_at_hospital"},
```

```
    {"agent_id": "1", "type": "act", "action_name": "extinguish_fire"},
    {"agent_id": "1", "type": "act", "action_name": "recharge"},
    {"agent_id": "5", "type": "act", "action_name": "clear_rubble"}
  ]
}
```

Valid `action_name`: `pickup_survivor`, `drop_at_hospital`, `extinguish_fire`, `clear_rubble`, `recharge`, `resupply`.
If malformed, increment `invalid_json` and re-prompt once.

5 Required Outputs

Logs (required)

Each tick's conversation must be stored in JSONL under:

```
logs/strategy=<name>/run=<id>/
  tick000.jsonl
  tick001.jsonl
  ...
```

Each JSONL contains lines like:

```
{"role":"system","content":"...system prompt..."}
{"role":"user","content":"...context JSON + tool specs..."}
{"role":"assistant","content":"Thought: ..."}
{"role":"assistant","content":"FINAL_JSON: {\"commands\":[...]}"}
```

Per-run JSON (stdout; save to results/raw/)

```
{
  "rescued": 20,
  "deaths": 5,
  "avg_rescue_time": 12.4,
  "fires_extinguished": 30,
  "roads_cleared": 4,
  "energy_used": 75,
  "tool_calls": 56,
  "invalid_json": 1,
  "replans": 2,
  "hospital_overflow_events": 1
}
```

Aggregated CSV (results/agg/summary.csv)

One row per run; columns:

```
run_id,map,strategy,seed,ticks,rescued,deaths,avg_rescue_time,
fires_extinguished,roads_cleared,energy_used,tool_calls,invalid_json,
replans,hospital_overflow_events
```

Plots (results/plots/)

- Bar: rescued and deaths by strategy × map.
- Line: cumulative rescued over ticks.
- Box: average rescue time per strategy.

GUI Screenshots

Initial, mid-run, and termination for at least one run per strategy.

6 Experiment Protocol

Run **3 planners × 3 maps × 5 seeds = 45 runs (minimum)**. Save *all* logs, JSON, CSV, plots, and screenshots. Your report must include quantitative metrics/plots and qualitative analysis of LLM behavior.

7 Report Format (6–8 pages)

Title Page (0.5 p) Title, author name, date.
Abstract (0.5 p; 150–200 words) Problem, methods (3 frameworks + LLM + env/GUI extensions), key results.
1. **Introduction (1 p)** Motivation for agentic AI in disaster response; why LLM-based (vs. rule-based).
2. **System Overview (1 p)** Provided vs implemented; pipeline diagram: *World → Sensors → LLM Planner → JSON Commands → World*.
3. **Methods (2–3 pp)** Three planners (flow/pseudocode, prompt templates, stopping criteria, schema enforcement & invalid-JSON handling); LLM API & parameters; environment extensions; GUI changes.
4. **Results (2 pp)** Tables (aggregated metrics), required plots, screenshots.
5. **Discussion (1–1.5 pp)** Compare frameworks; analyze invalid JSON, replans, overflow; effect of environment constraints.
6. **Conclusion (0.5 p)** Findings, limitations, and future work.

References Cite frameworks/tools used.
Appendix (optional) Extra prompts/logs/plots.

8 Grading Rubric (100 Marks)

Category	Sub-Category	Marks	Evidence / Requirements
Implementation (50)	3 LLM planners	18	3×6: Any of {ReAct, Reflexion, Plan&Execute, CoT, ToT}. Must be LLM-driven (no rule-based planning).
	ReAct baseline upgrade	2	Replace mock with proper LLM ReAct loop (thought→observation→final JSON).

Category	Sub-Category	Marks	Evidence / Requirements
	Environment extensions	15	3×5: Battery/recharge; Medic slowdown; one of {Rubble clearing, Triage policy, Aftershocks}. Properly integrated in `dynamics.py`/`resources.py`.
	GUI upgrades	15	3×5: Full legend; extended stats panel; charts that **stop** on termination.
LLM Integration (15)	Working API client	5	`llm_client.py` implements real provider (Groq/Gemini), env var keys, error handling.
	Prompt & schema enforcement	3	Prompts specify allowed actions & exact JSON schema; invalid JSON increments metric & triggers one re-prompt.
	Logging	2	JSONL prompt/response per tick per run in `logs/`.
	Example logs in report	2	At least one excerpt (system/user/assistant + FINAL_JSON) shown.
	Real runs demonstrated	3	CLI/GUI screenshots and metrics confirming execution.
Experiments & Analysis (20)	Coverage	6	3 maps × ≥5 seeds each across ≥3 strategies (min 45 runs).
	Tables of metrics	4	Aggregated tables with mean/stdev across seeds.
	Plots	4	Required bar/line/box plots saved in `results/plots/` and included in report.
	Screenshots	3	GUI initial, mid, end for at least one run/strategy.
	Analysis of failures	3	Discuss invalid JSON, replans, LLM hallucinations, overflow events.
Report Quality (15)	Abstract	2	150–200 words, clear summary.
	Structure	3	All required sections present in order.
	Clarity & depth	5	Formal style, figures/tables with captions, sensible interpretation.
	Reproducibility	5	Methods precise: versions, params, seeds, file paths.

Category	Sub-Category	Marks	Evidence / Requirements
Automatic Deductions	Rule-based planner	*Fail planners portion*	Any planner with rule-based decision logic (beyond JSON validation/safety) fails the planners portion.
	Missing artifacts	−5 each	Missing logs, plots, or screenshots: minus 5 per missing artifact.
	Late submission	−10%/day	Deducted from total after marking.

9 Submission Checklist (Quick)

- ☐ 3 LLM planners implemented (ReAct + any 2).
- ☐ `llm_client.py` calls Groq/Gemini; keys documented; JSONL logs saved.
- ☐ Environment extended with ≥3 features (battery, slowdown, triage/rubble/aftershocks).
- ☐ GUI: full legend, extended stats, charts that stop.
- ☐ Experiments: 3 maps, ≥5 seeds each, ≥3 strategies.
- ☐ Saved per-run JSONs (`results/raw/`), aggregated CSV (`results/agg/summary.csv`), required plots (`results/plots/`).
- ☐ 6–8 page PDF report with all sections & assets.

Notes (Read Before You Start)

- Decision-making must be LLM-driven; you may validate schemas and enforce safety, but not *decide* plans via rules.
- Keep `sensors.py` context bounded (nearest-k items, summaries) to control tokens/cost.
- Cap per-tick ReAct steps to limit latency; use Reflexion memory to reduce repeat errors.
- Termination: `rescued` + `deaths` = `total_survivors` **OR** $t \geq$ `max_ticks`. Charts must stop thereafter.